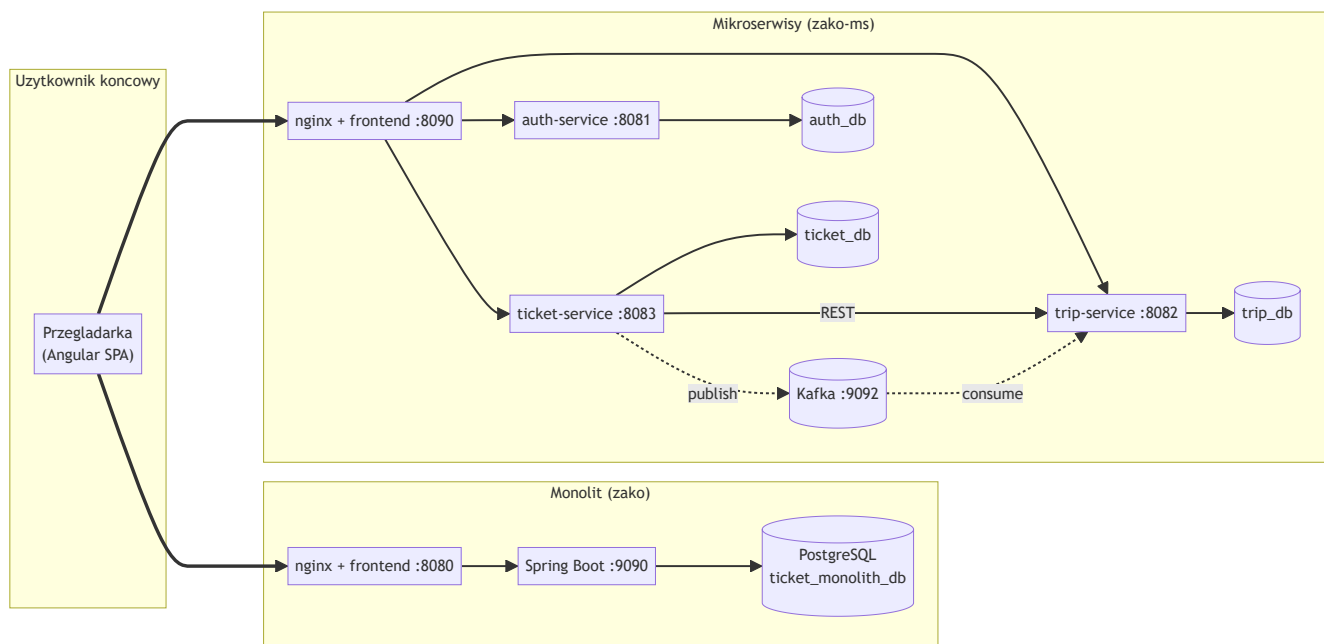


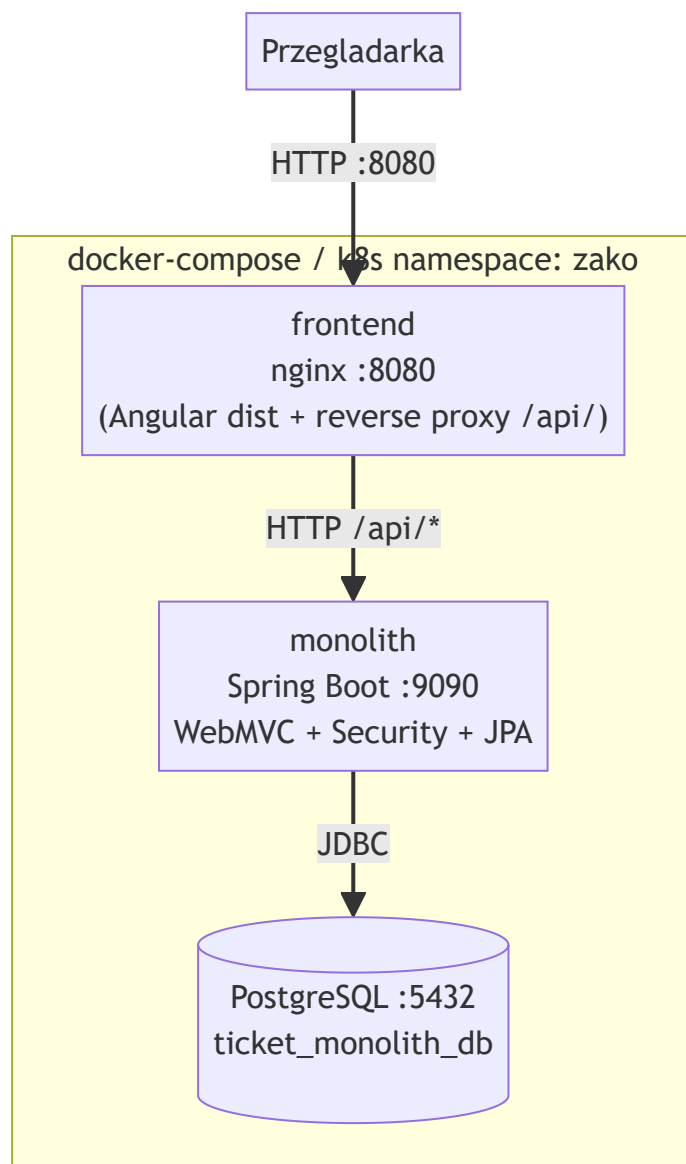
Zako — Architektura porównawcza: Monolit vs Mikroserwisy

Dokument zawiera komplet diagramów obu wariantów wdrożenia aplikacji **Zako** (rezerwacja biletów kolejowych): wersji monolitycznej (Spring Boot na pojedynczej bazie Postgres) oraz wersji mikroserwisowej (auth + trip + ticket + Kafka).

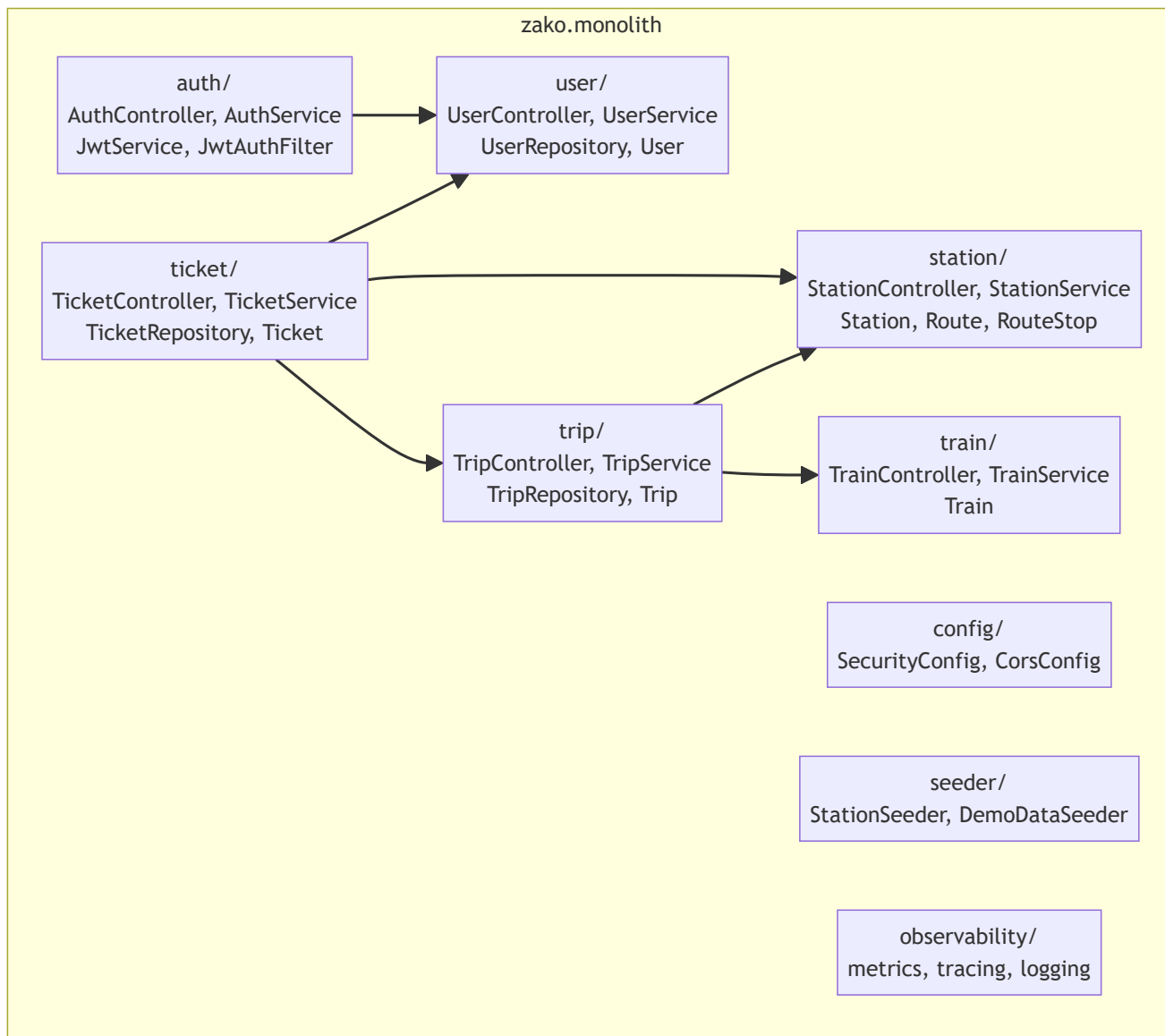
1. Kontekst systemu



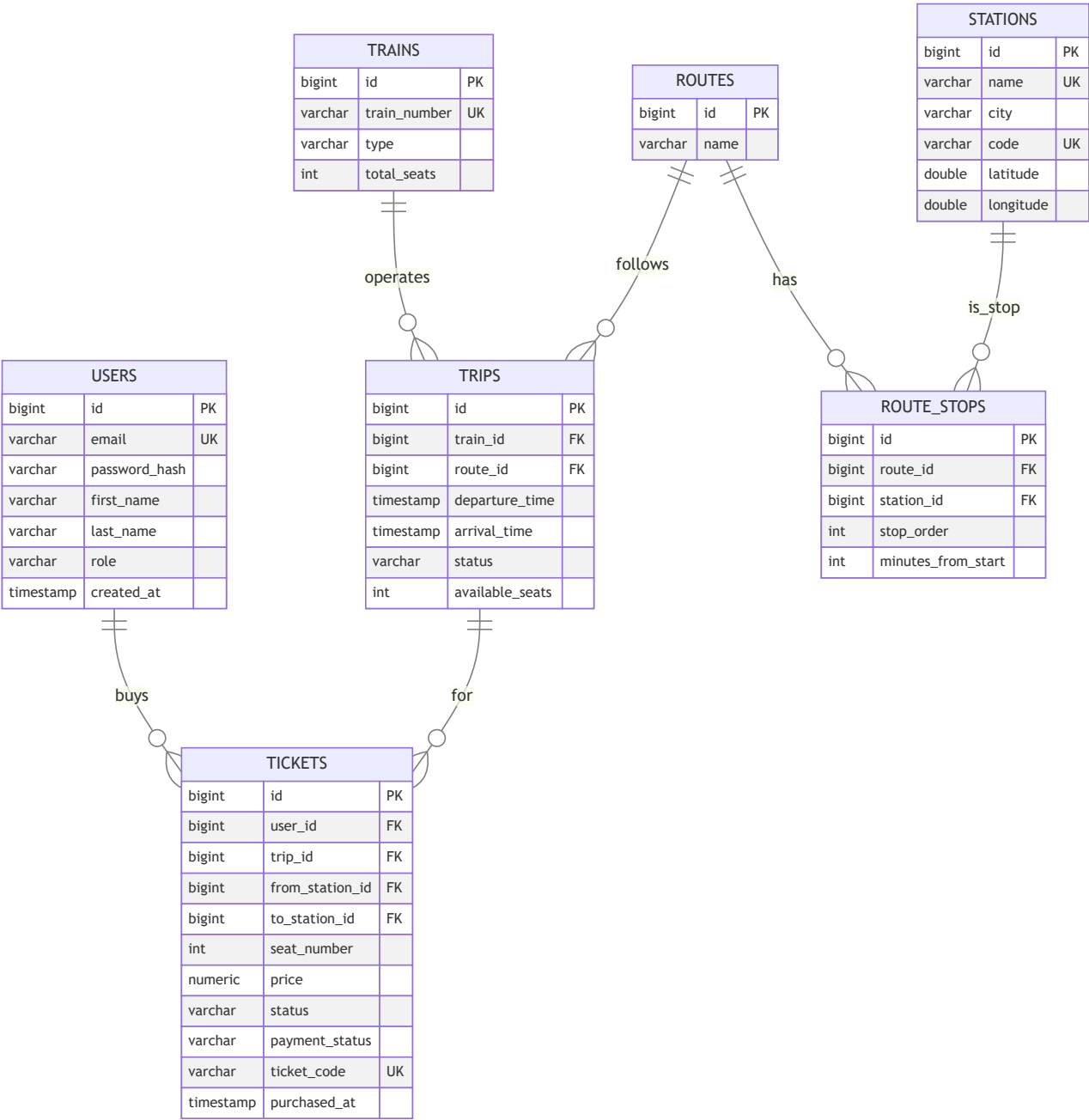
2. Monolit — diagram kontenerow



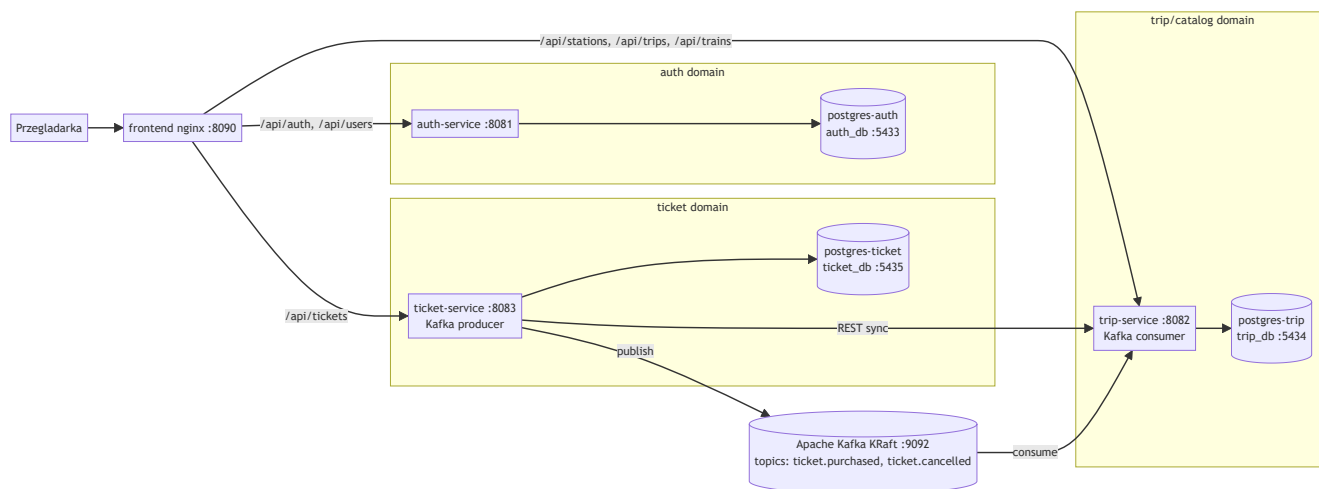
2.1 Pakiety / komponenty monolitu



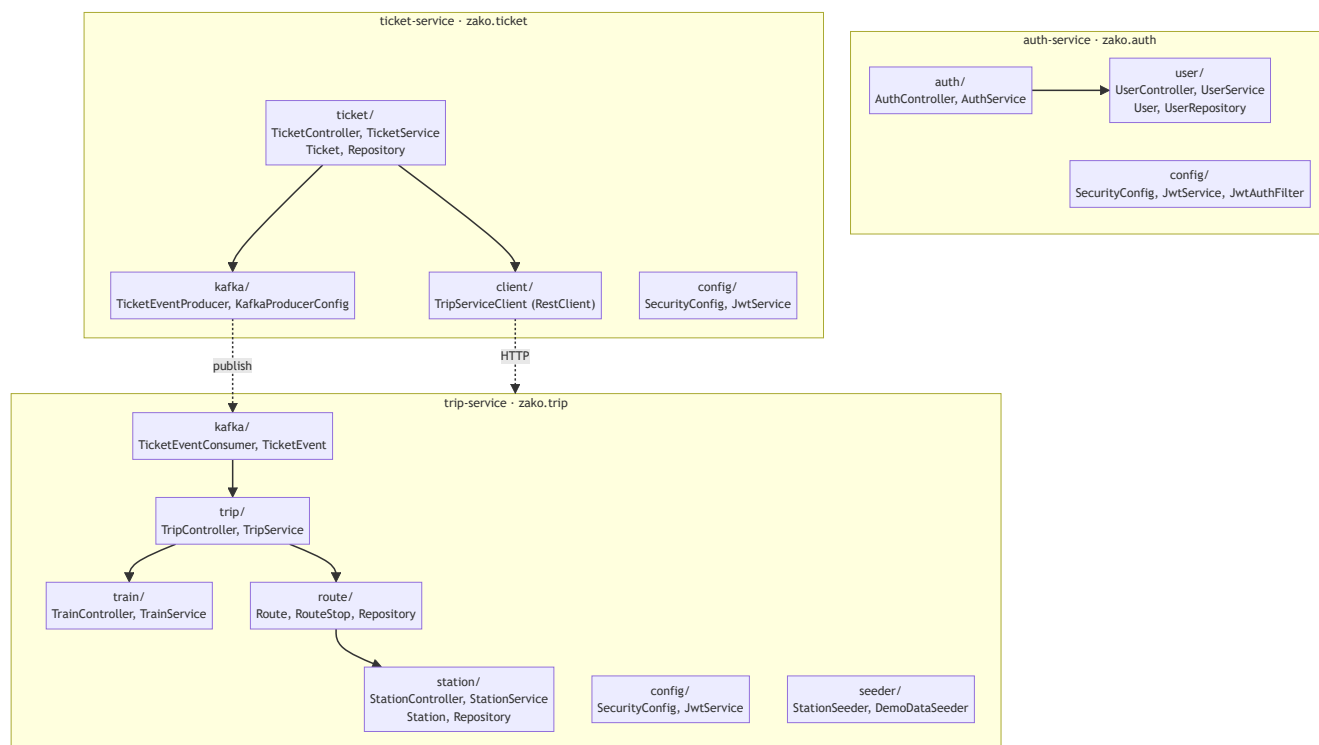
2.2 Model danych monolitu — ER



3. Mikroserwisy — diagram kontenerow



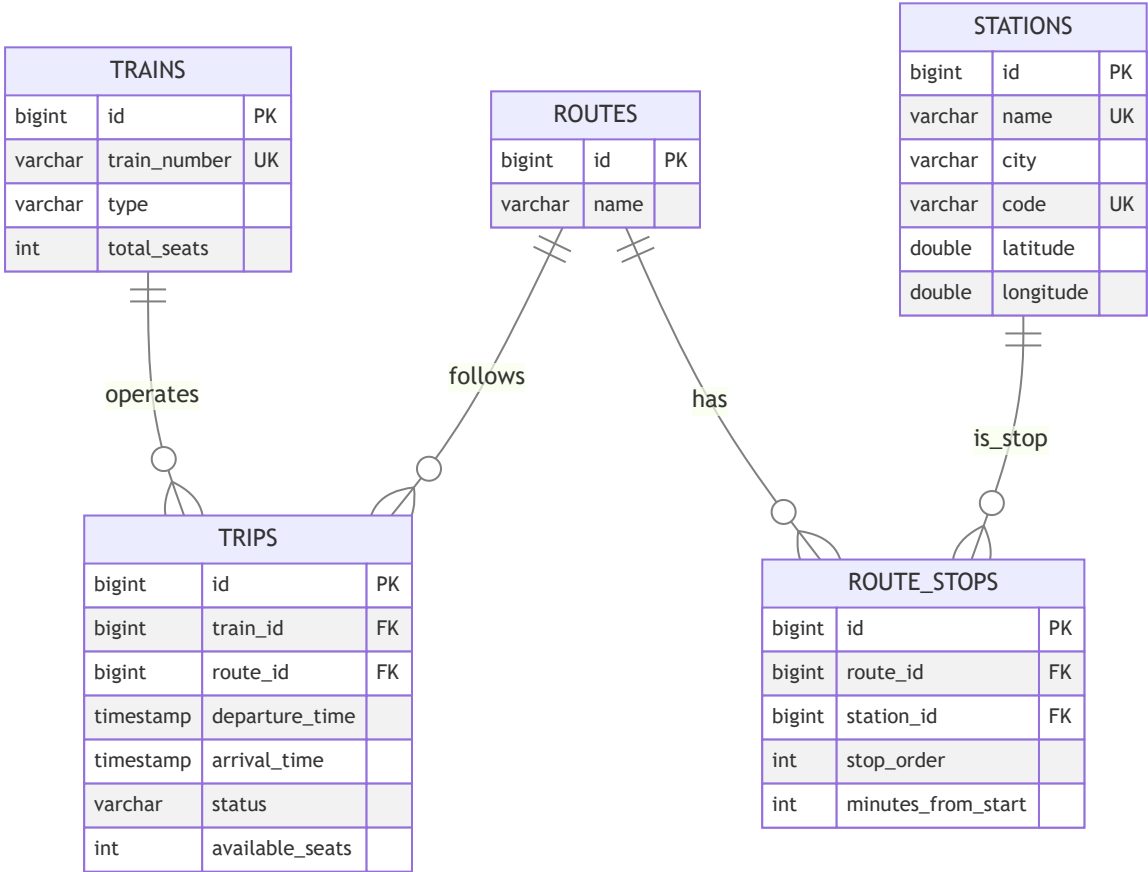
3.1 Pakiety / komponenty mikroserwisow



3.2 ER auth_db

USERS		
bigint	id	PK
varchar	email	UK
varchar	password_hash	
varchar	first_name	
varchar	last_name	
varchar	role	
timestamp	created_at	

3.3 ER trip_db



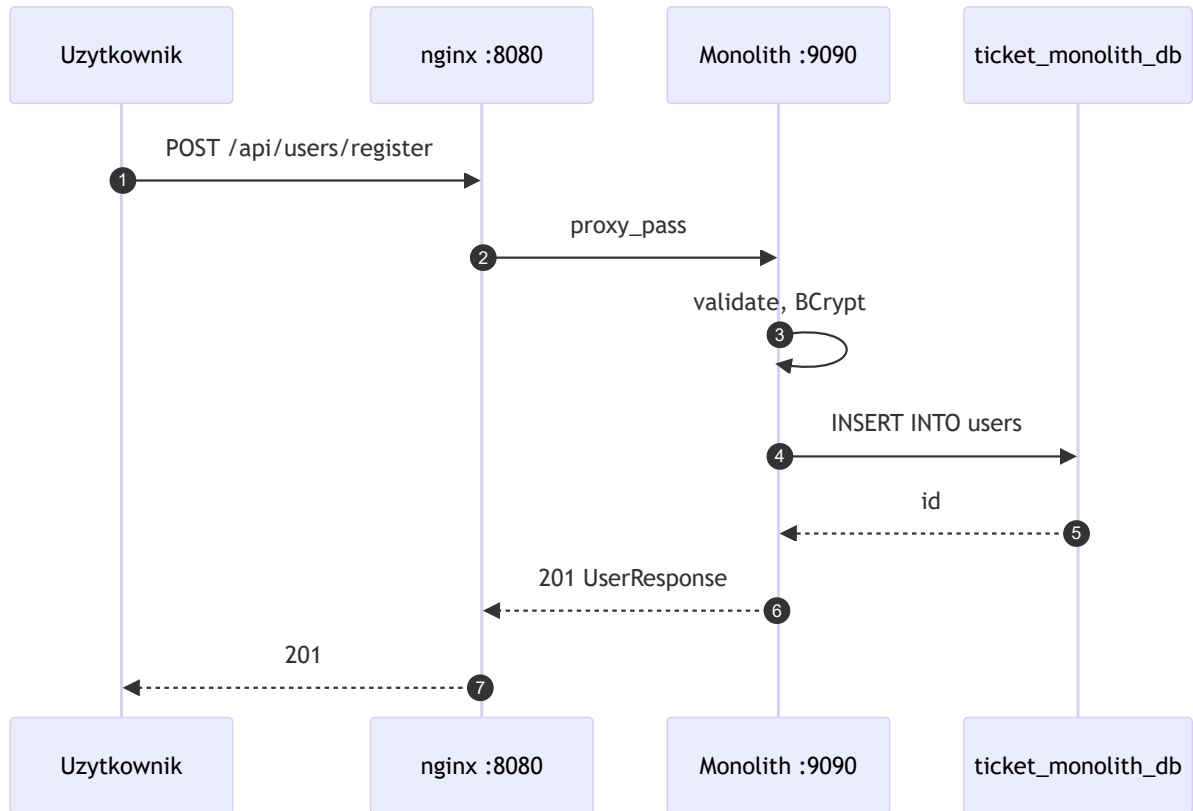
3.4 ER ticket_db

TICKETS			
bigint	id	PK	
bigint	user_id		no FK
bigint	trip_id		no FK
bigint	from_station_id		no FK
varchar	from_station_name		denorm
varchar	from_station_city		denorm
varchar	from_station_code		denorm
bigint	to_station_id		no FK
varchar	to_station_name		denorm
varchar	to_station_city		denorm
varchar	to_station_code		denorm
int	seat_number		
timestamp	departure_time		denorm V2
timestamp	arrival_time		denorm V2
numeric	price		
varchar	status		
varchar	payment_status		
varchar	ticket_code	UK	
timestamp	purchased_at		

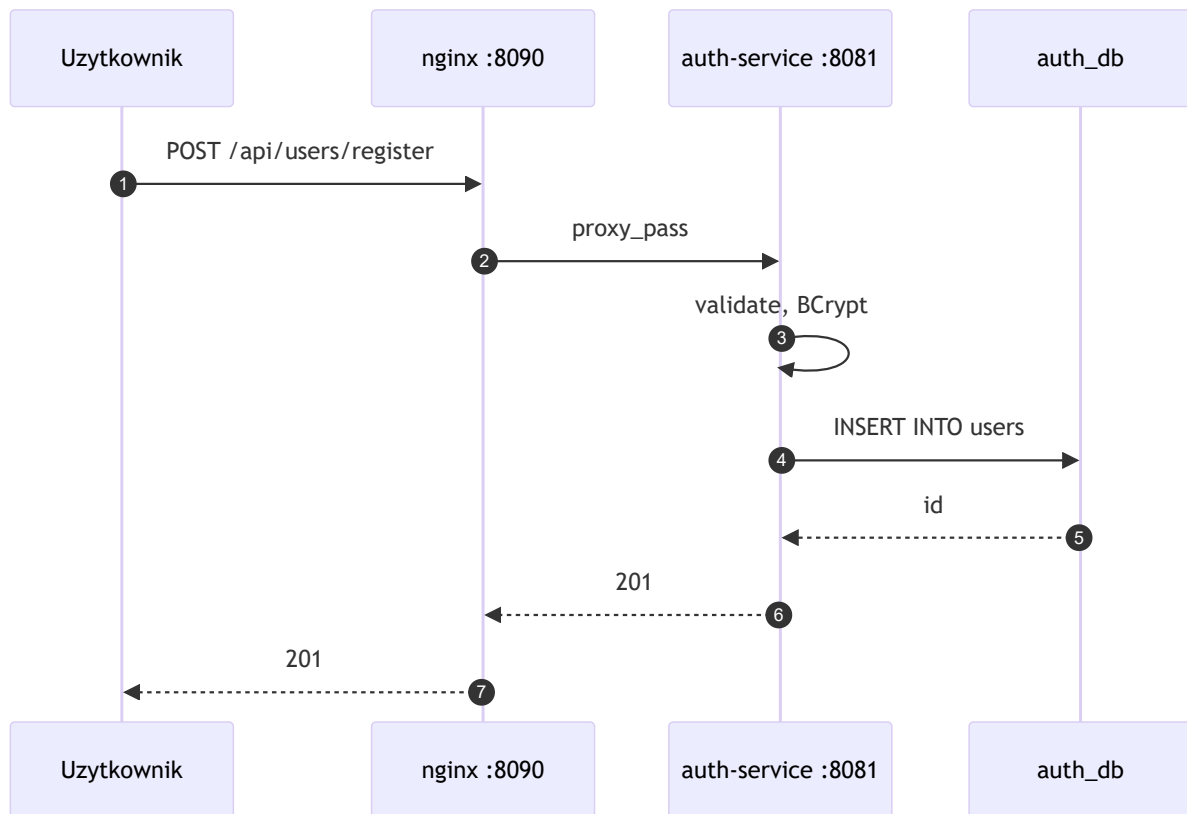
Pojedyncza tabela tickets; obce identyfikatory wyłączenie jako liczby, dane z innych domen denormalizowane w momencie zakupu.

4. Sekwencja — rejestracja

4.1 Monolit

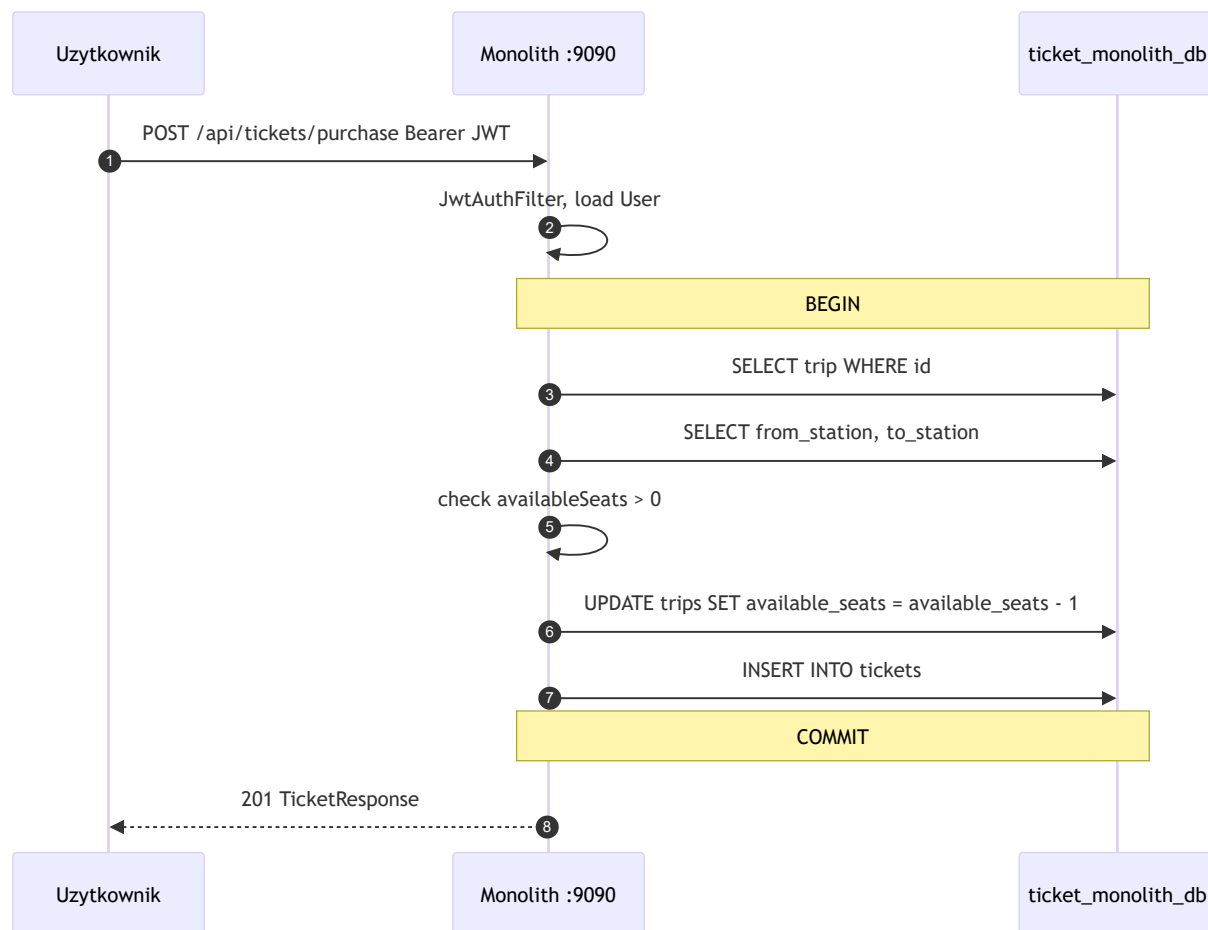


4.2 Mikroserwisy

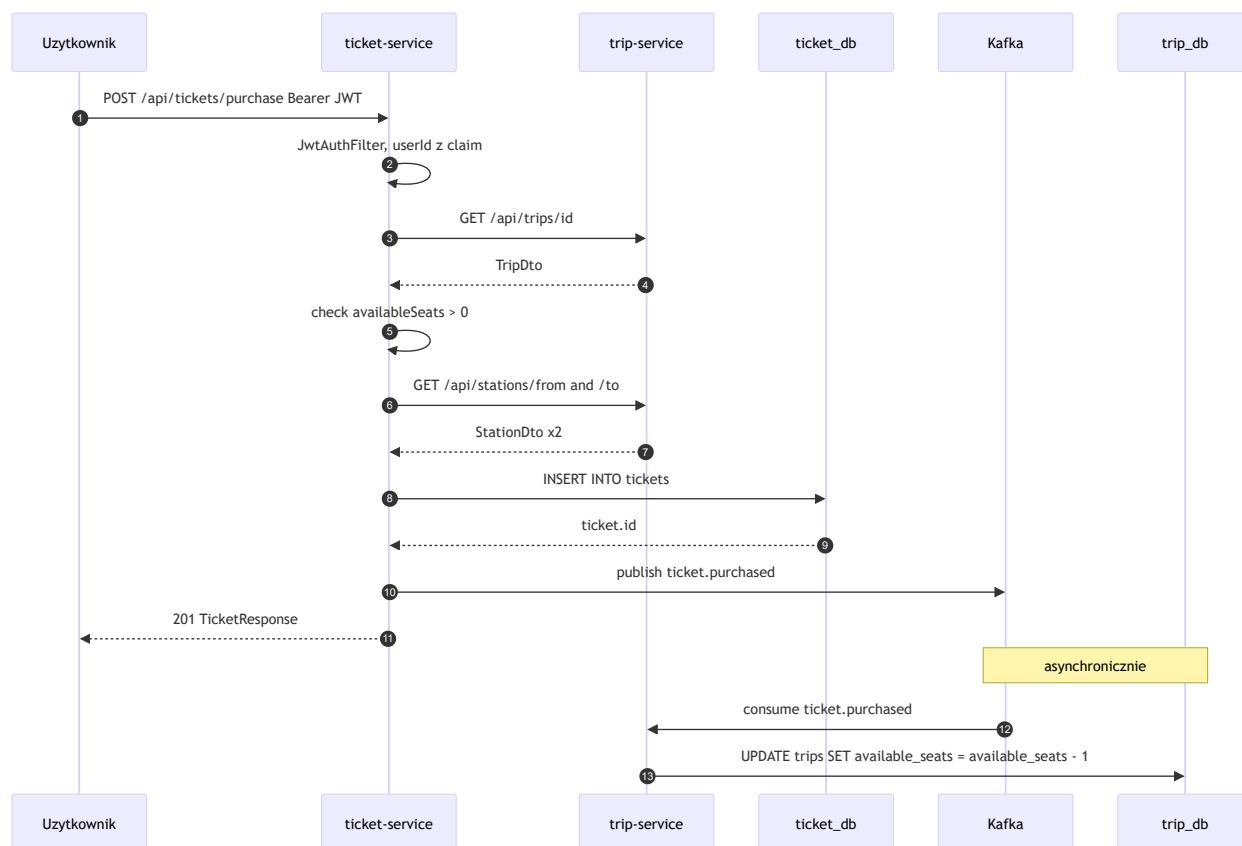


5. Sekwencja — zakup biletu

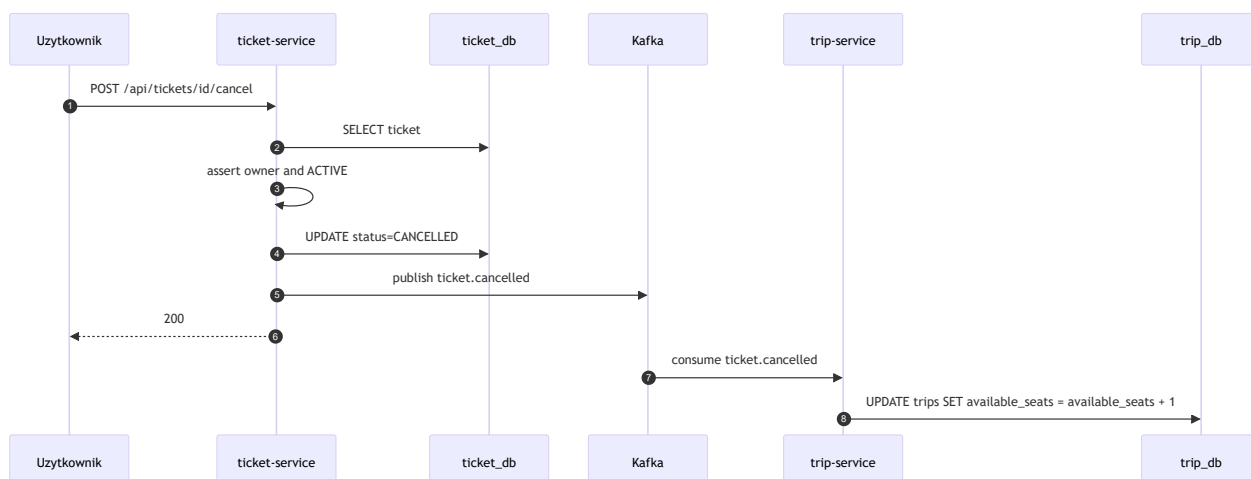
5.1 Monolit (jedna transakcja)



5.2 Mikroserwisy (REST + Kafka, eventual consistency)

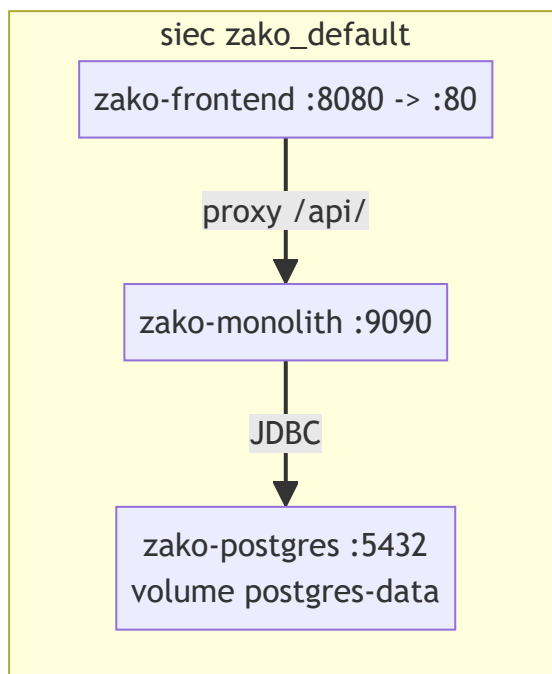


5.3 Anulowanie biletu (mikroserwisy)

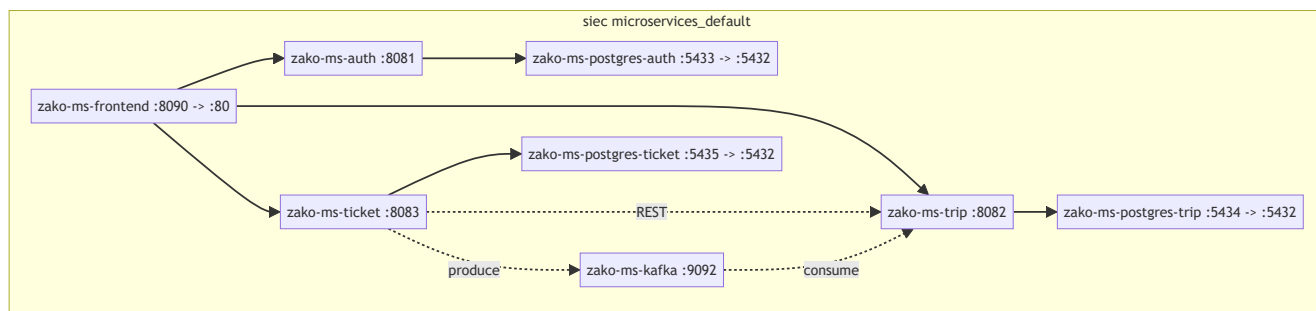


6. Wdrozenie — docker-compose

6.1 Monolit

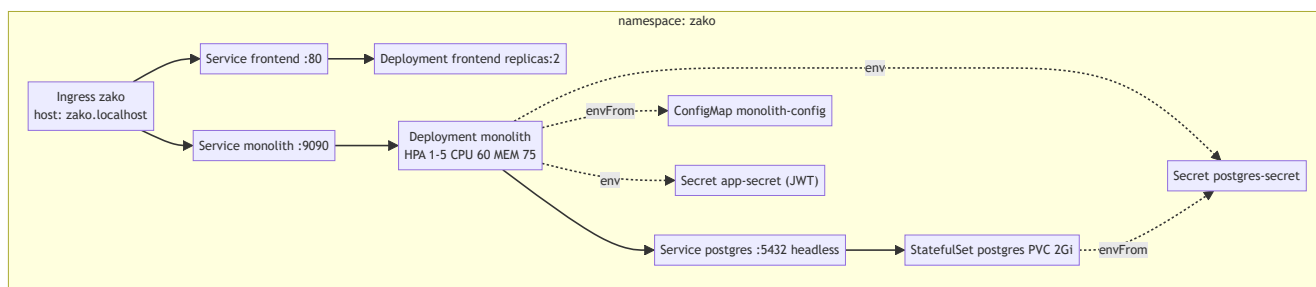


6.2 Mikroserwisy

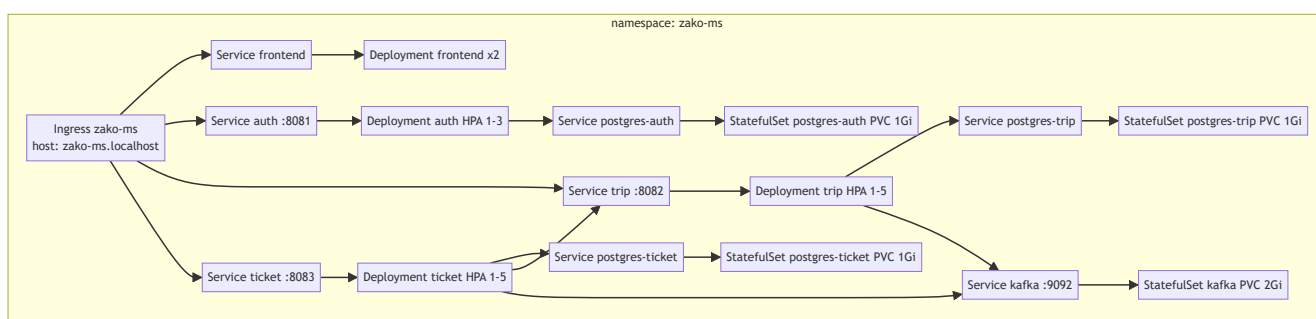


7. Wdrozenie — Kubernetes

7.1 Namespace zako (monolit)



7.2 Namespace zako-ms (mikroserwisy)



8. Porównanie

Aspekt	Monolit	Mikroserwis
Procesy JVM	1	3 (auth + trip + ticket)
Bazy Postgres	1 (ticket_monolith_db)	3 (auth_db, trip_db, ticket_db)
Schemat danych	Wspólny, FK między tabelami	Rozdzielony, brak FK między bazami
Komunikacja modułów	wywołania w obrębie JVM (@Service)	REST sync (ticket -> trip) + Kafka async
Transakcyjność	Lokalna @Transactional obejmuje wszystkie domeny	Lokalna w obrębie serwisu; spójność seatów: eventual consistency przez Kafkę
Autoryzacja	Ten sam SecurityFilterChain dla /api/*	Każdy serwis ma własny JwtAuthFilter z tym samym sekretem
Wdrożenie (k8s)	1 Deployment + 1 StatefulSet	3 Deployment + 4 StatefulSet (3xPostgres + Kafka)
Skalowanie	HPA jednego Deploymentu	HPA na każdy serwis (trip 1-5, ticket 1-5, auth 1-3)
Awaria	Awaria procesu = brak całej funkcjonalności	Awaria ticket-service nie blokuje logowania i przeglądania
Czas startu	~14 s	3 x ~25-30 s + Kafka

9. Wybrane decyzje projektowe

1. **Denormalizacja stacji + czasów w tickets** (mikroserwis). Bilet po sprzedaży nie zależy od późniejszych zmian w trip_db.
2. **Identyfikatory bez FK** między bazami. Ticket trzyma user_id, trip_id, station_id jako liczby; weryfikacja przez REST przy zakupie.
3. **Kafka KRaft single-node** zamiast klastra z ZooKeeperem — wystarczające dla demo.
4. **CORS** — monolit używał setAllowedOrigins("http://localhost:4200"); zmienione na setAllowedOriginPatterns dla localhost:* i ingress hostów.
5. **spring-boot-flyway starter** w ticket-service — flyway-core sam z siebie nie aktywuje autoconfig w Spring Boot 4.0, migracja V1 była pomijana.